

TÀI LIỆU KIẾN TRÚC CHÍNH THỨC

EGAL-CED — CENTRALIZED ERROR DOCTRINE

v1.1.0 • APPROVED & FROZEN (Ops-hardened)

Kế thừa v1.0.0 + phản biện thực chiến (observability, async, registry, i18n-ready)

Trạng thái	FROZEN v1.1.0 — Implementation-Ready foundation
Changelog	E5 correlation BẮT BUỘC ở Global Handler; VIII.1 Observability; VIII.2 Async; VII registry domain + CI uniqueness; V.4 i18n-ready details; logPayload tách external wire
Map	EGAL-SEC v1.1 • OPD v1.2 • KOS • Impl Plan SEC/CED phases

Tài liệu này thay thế CED v1.0.0 làm SSOT. v1.0 giữ lịch sử. Phản biện thực chiến được tiếp thu có chọn lọc — không phá dual-contract / Q1 message UAT.

0. Tiếp thu phản biện (v1.0 → v1.1)

0.1. Giữ nguyên (Pros)

- E2 Operational vs Programmer — nền tảng alerting.
- E4 HTTP 423 Locked — state machine FE.
- E7 Dual-contract + aliases — zero breaking FE.
- Path shared/errors — platform kernel, không auth-local.

0.2. Vá theo critique

Critique	Xử lý v1.1
Registry contention / circular	Domain files độc lập (đã có); index chỉ aggregate freeze; CI test unique values; cấm domain import service qua errors; import codes-only path
Log / cause.stack mất	§VIII.1 Internal Log Payload bắt buộc serialize stack + cause chain; tách external wire
correlationId optional	E5 nâng cấp: BẮT BUỘC tại Global Handler (sinh UUID nếu thiếu)
Express async unhandled	§VIII.2 asyncHandler / express-async-errors bắt buộc CED-2
i18n message cứng	§V.4 details.i18nKey + i18nParams (opt-in); message vẫn fallback UAT string

I. Mục tiêu & phạm vi

CED = kernel tạo / phân loại / serialize / log lỗi toàn backend. v1.1 thêm Observability contract và Async guarantee.

- Trong: AppError, registry domain-scoped, wire, dual, log payload, async boundary, i18n-ready details.
- Ngoài: full i18n catalogs; Sentry vendor lock; OpenAPI codegen (extension).

II. Invariants (E1–E9)

- E1. E1 — Stable Code: rename chỉ dual alias + deprecation.
- E2. E2 — Operational vs Programmer: isOperational; non-op → alert channel.
- E3. E3 — Single Canonical Wire: success + error object.
- E4. E4 — Status Honesty: 423 lifecycle/workflow; không 500 cho business deny.
- E5. E5 — Correlation Mandatory at edge: Global Handler LUÔN gắn correlationId (req → header → new UUID). Service nên truyền; handler không được omit.
- E6. E6 — Namespace Ownership: domain files; không đụng code namespace khác.
- E7. E7 — Dual Contract (SEC E): primary + legacy alias table.
- E8. E8 — Human Authority: approval/deny codes rõ.
- E9. E9 — Dual Output: External Response ≠ Internal Log. Log giữ stack/cause; Response không lộ PII/stack production.

III. Taxonomy & HTTP

Giữ v1.0: AUTH / SEC / TENANT / ONBOARDING / COMMON / domain. HTTP map: 401/403/404/409/422/423/429/500 — 423 Locked cho CHO_DUYET, tenant pending, onboarding wrong state.

IV. AppError model

```
class AppError extends Error {
  code, statusCode, details?, correlationId?, cause?, isOperational=true
  name: 'AppError' | 'BusinessError'
}
// details (optional i18n-ready):
// { currentStatus, caseId, i18nKey?, i18nParams?, ...safe metadata }
```

- cause: native/DB error — KHÔNG đưa raw cause.message nhạy cảm ra client; CHỈ internal log.
- BusinessError alias giữ Q1 onboarding.

V. Wire contract

V.1. Canonical (external)

```
{
  "success": false,
  "error": {
    "code": "ONBOARDING_CASE_NOT_EDITABLE",
    "message": "Hồ sơ không được chỉnh sửa.",
    "details": {
      "currentStatus": "SUBMITTED",
      "i18nKey": "errors.onboarding.not_editable",
      "i18nParams": { "status": "SUBMITTED" }
    },
    "correlationId": "<uuid-mandatory>"
  }
}
```

V.2. Dual legacy

- toResponse({ legacy:true }) thêm top-level code/message khi cần FE cũ.

V.3. Success

- { success:true, data } — CED không bắt buộc đổi success path.

V.4. i18n-ready details (v1.1)

- message: luôn có fallback tiếng Việt (UAT) cho client chưa i18n.
- details.i18nKey / i18nParams: optional; FE/phase sau localize.
- Không bắt buộc mọi throw có i18nKey ở v1.1 — khuyến nghị lỗi user-facing mới.
- Cấm PII (password, OTP, full token) trong details.

VI. Registry — domain scoping (v1.1 chống contention)

VI.1. Layout

```
shared/errors/codes/
  auth.codes.js      // chỉ AUTH namespace object
  onboarding.codes.js
  tenant.codes.js
  security.codes.js
  common.codes.js
  index.js           // aggregate Object.freeze — KHÔNG logic

// Import khuyến nghị (tránh circular):
const { ONBOARDING } = require('../../shared/errors/codes/onboarding.codes');
// hoặc barrel: require('../../shared/errors').ERROR_CODES.ONBOARDING
```

VI.2. Quy tắc chống circular & conflict

- codes/*: zero import từ modules/* hoặc services/* (chỉ constants).
- Domain PR: ưu tiên chỉ định file namespace của domain; index.js chỉ thêm 1 dòng require nếu namespace mới.
- CI (bắt buộc khi CED-1 merge): test không trùng string value giữa mọi namespace (flatten values unique).
- Không dùng Proxy runtime bắt buộc — freeze object đủ; Proxy optional DX sau.

VI.3. Dual alias seed

Giữ bảng v1.0: ACCOUNT_CHO_DUYET ↔ AUTH_ACCOUNT_CHO_DUYET, v.v. Primary = mã FE live.

VII. Boundary (throw / serialize)

- Service: throw `AppError`; không `res.json`.
- Controller: `try/catch` + `sendError` HOẶC `next(err)` với `asyncHandler`.
- Middleware: `next(AppError)` ưu tiên; hoặc `sendError`.
- Global handler: dual output E9 + E5 correlation.

VIII. Observability & Async (v1.1 normative)

VIII.1. Dual output — External vs Internal Log

Global Handler / `sendError` thực hiện HAI việc tách biệt:

1. External Response = `toResponse(err)` — canonical; production không stack; không cause raw.
2. Internal Log = `toLogPayload(err, req)` — luôn có:
 - `correlationId` (mandatory)
 - `code`, `statusCode`, `message`, `details` (redacted)
 - `err.stack` (operational: debug/info; non-op: error level)
 - `cause chain`: `cause.message`, `cause.stack`, `cause.code` (pg/mongo)
 - `request`: `method`, `path`, `userId?`, `tenantId?`, `ip` (không body password)
 - `isOperational` flag → routing alert (Sentry only if `!isOperational` hoặc `status>=500` non-expected)

toLogPayload shape (normative)

```
{
  level: "warn" | "error",
  correlationId: "uuid",
  code: "ONBOARDING_MERGE_FAILED",
  statusCode: 500,
  message: "...",
  isOperational: true,
  details: { ... },
  stack: "...",
  cause: { message, name, stack, code? },
  request: { method, path, userId, tenantId, ip },
}
```

- `JSON.stringify` log: dùng serializer an toàn (stack string); không dựa vào default `Error` enumerable.

VIII.2. Express Async Error Handling (bắt buộc CED-2)

- Express 4: throw trong async function KHÔNG vào middleware lỗi trừ khi:
 - `asyncHandler(fn)` wrapper: `Promise.resolve(fn(req,res,next)).catch(next)`
 - HOẶC `require('express-async-errors')` sớm trong app bootstrap
 - HOẶC mọi controller `try/catch` + `next(err)` (để sót — không đủ một mình)
- CED-2 exit criteria: 100% route async đi `asyncHandler` hoặc library; test throw `AppError` trong async → đúng status không crash.
- Skeleton: `shared/errors/asyncHandler.js`.

IX. Map SEC / OPD / KOS

- SEC E = CED wire + dual + E5 correlation trên mọi security deny.
- OPD ONBOARDING_* + 423; MERGE_FAILED operational 500 + log cause.
- KOS: stable codes; Working Context correlation; Human Authority codes rõ.

X. Filesystem skeleton (v1.1)

```
src/shared/errors/  
  index.js  
  AppError.js  
  createError.js  
  toResponse.js  
  toLogPayload.js      // NEW v1.1  
  asyncHandler.js      // NEW v1.1  
  httpMap.js  
  aliases.js  
  codes/{auth,common,onboarding,security,tenant,index}.js  
  README.md
```

XI. Adoption phases (cập nhật)

- CED-1: Skeleton v1.1 + CI unique-codes test script.
- CED-2: Wire global handler (toResponse + toLogPayload + E5 UUID) + asyncHandler|express-async-errors.
- CED-3: auth re-export codes; login.core createError.
- CED-4: onboarding BusinessError từ shared; controller sendError.
- CED-5: middleware unify; i18nKey dần cho user-facing mới.

Song song SEC 1A OK; CED-2 nên trước/cùng SEC 1B.

XII. Anti-patterns

- Gom logic domain vào codes/index.js.
- Import auth.service từ codes (circular).
- Log password/OTP; đưa cause.stack ra client production.
- Bỏ qua asyncHandler rồi tin global handler “tự bắt”.
- Thiếu correlationId trên response sau CED-2.

XIII. Freeze checklist v1.1

- E1–E9 locked (E5 mandatory, E9 dual output).
- VIII.1 log contract + VIII.2 async locked.
- Registry domain files + CI uniqueness rule locked.
- i18nKey/i18nParams convention locked (optional fields).
- Skeleton v1.1 ship kèm; app wire = CED-2.

XIV. Kết luận

v1.0 vững về contract API. v1.1 đóng góc khuất vận hành: log/cause, correlation bắt buộc, async Express, registry scale, i18n-ready — đủ Implementation-Ready khi đi CED-2+.

Bản hiệu chỉnh **EGAL-CED v1.1.0** này đã hoàn toàn **chín chắn, chặt chẽ và cực kỳ thực chiến**. Các điểm yếu ở góc khuất vận hành (Express Async, Circular Dependencies, Loss of Stack/Cause Trace, i18n Tech Debt, Correlation ID gap) đều đã được giải quyết bằng các quy tắc kiến trúc chuẩn mực và có tính ràng buộc cao (Normative Rules).

Dưới đây là phần **Review chuyên sâu** về bản v1.1.0 cùng **Checklist kiểm định triển khai (Execution Audit Checklist)** để giúp bạn và đội ngũ Dev/Ops áp dụng trơn tru trong giai đoạn CED-1 đến CED-5:

I. ĐÁNH GIÁ CÁC ĐIỂM CẢI TIẾN TRỌNG TÂM TRONG v1.1.0

1. Chuẩn hóa Tách biệt Dual Output (E9 & VIII.1):

- **Điểm cộng:** Việc phân tách rõ ràng giữa `toResponse(err)` (cho Client - an toàn, không lộ thông tin nhạy cảm) và `toLogPayload(err, req)` (cho Ops/Dev - giữ trọn `err.stack`, `cause.stack`, `userId`, `correlationId`) giải quyết triệt để bài toán Observability mà vẫn bảo mật PII.
- **Đặc biệt:** Quy định `serialize cause` từ database (Postgres/Mongo) là cực kỳ thực chiến, giúp truy vết tận gốc lỗi root cause khi debugging production.

2. Cơ chế Async Error Handling Guarantee (VIII.2):

- **Điểm cộng:** Việc bắt buộc tích hợp `asyncHandler` hoặc `express-async-errors` ngay từ **Phase CED-2** là yếu tố quyết định để tránh việc Express 4 "nuốt lỗi" Async Promise Rejection hoặc làm crash process ngoài ý muốn.

3. Cơ chế Chống Circular Dependency & Collision ở Registry (VI.1 - VI.2):

- **Điểm cộng:** Tách registry thành từng domain file độc lập (`auth.codes.js`, `onboarding.codes.js...`) và quy định **zero import từ business logic** giúp tiêu diệt nguy cơ circular dependency.
- **Điểm cộng:** Bổ sung **CI Test tự động kiểm tra Uniqueness của Code values** là giải pháp phòng ngừa tuyệt vời cho làm việc nhóm (Team Collaboration).

4. i18n-Ready với Backward Compatibility (V.4):

- **Điểm cộng:** Cấu trúc `i18nKey` và `i18nParams` nằm trong `details` giúp FE có thể chủ động chuyển sang đa ngôn ngữ bất cứ lúc nào mà không làm gãy contract message tiếng Việt hiện tại đang dùng cho UAT/Legacy FE.

5. Correlation ID Bắt buộc tại Edge (E5):

- **Điểm cộng:** Chuyển `correlationId` từ *Optional* sang *Mandatory at Global Handler* nâng tầm hệ thống lên chuẩn Enterprise Tracing (chuẩn bị sẵn sàng cho Distributed Tracing / OpenTelemetry nếu tương lai mở rộng sang Microservices).

II. CHECKLIST KIỂM ĐỊNH THỰC CHUYỂN CHO ĐỘI NGŨ DEV/OPS (CED IMPLEMENTATION AUDIT)

Khi triển khai các phase (CED-1 đến CED-5), bạn có thể dùng Bảng Checklist này để review Code / PR của Developers:

1. Tại Shared Kernel (**src/shared/errors/**) — Phase CED-1:

- ☐ `toLogPayload.js` có hàm `serializer` an toàn xử lý được thuộc tính `stack` (vì thuộc tính `stack` của native `Error` trong V8 mặc định là *non-enumerable*, `JSON.stringify(err)` sẽ ra `{}` nếu không lấy `err.stack` thủ công).
- ☐ `toLogPayload.js` đã `redact/masking` các header hoặc field nhạy cảm (VD: `authorization`, `password`, `otp`, `token`).
- ☐ Script CI (VD: `npm run test:error-codes`) được viết để `flatten` toàn bộ string value trong các file `shared/errors/codes/*.codes.js` và đảm bảo **không có 2 hàng số nào trùng giá trị string**.

2. Tại Tầng Middleware & Global Handler (**app.js**) — Phase CED-2:

- [] Tích hợp **express-async-errors** ở ngay dòng đầu tiên của **app.js** (hoặc bọc toàn bộ Controller Routes bằng **asyncHandler**).
- [] Global Error Handler luôn gắn **correlationId** vào **res.setHeader('X-Correlation-Id', correlationId)** và đưa vào JSON response.
- [] Test thử nghiệm (Integration Test):
 - [] Throw thử một **AppError(isOperational=true)** trong 1 async route → Trả về đúng HTTP status code (vd 423), log ở level **WARN**.
 - [] Throw thử một **TypeError** (hoặc syntax error - non-operational) → Trả về **500 INTERNAL_ERROR** trên response, log ở level **ERROR** kèm full stack trace.

3. Tại Tầng Domain/Service — Phase CED-3 & CED-4:

- [] Cấm tuyệt đối việc import **service** hay **controller** vào các file ***.codes.js**.
- [] Mọi code mới khi **throw** phải dùng hằng số từ **ERROR_CODES**, không được hard-code string literals (VD: **throw new AppError(ERROR_CODES.AUTH.INVALID_CREDENTIALS, ...)**).
- [] Kiểm tra xem các lệnh **catch** từ DB driver (Kysely/Prisma/Knex/Mongoose) có truyền **cause** gốc vào **AppError** hay không:

// Chuẩn:

try

 await

 catch

 throw new

hồ sơ thất bại'

 cause

// Để toLogPayload ghi nhận được native DB error

'Lưu

...

III. KẾT LUẬN

Tài liệu **EGAL-CED v1.1.0** hiện tại đã **đạt chuẩn Architecture Gold Standard**:

- **Lý thuyết kiến trúc vững chắc** (phân định rõ Operational vs Non-operational, Canonical vs Dual Contract)[cite: 1, 2].
- **Thực chiến danh thép** (Async express boundary, Log vs Response separation, CI uniqueness enforcement).

Tài liệu này hoàn toàn đủ điều kiện **FROZEN & IMPLEMENTATION-READY**. Bạn có thể tự tin bàn giao bản spec này cho Tech Lead / Backend Team để tiến hành viết code cho Skeleton Phase CED-1 và Wiring Phase CED-2.